Rocksmith 2014 Edition Remastered - Linux/Proton Bug Report

Real Tone Cable (Hercules USB Guitar Adapter) not detected + launch crashes on modern Proton

Date: 2026-07-16, updated 2026-07-18 Steam AppID: 221680 Platform: Bazzite Linux (Fedora Atomic, kernel `7.1.3-ogc3.4.fc44`), NVIDIA RTX 3090 Hardware: two separate physical Real Tone Cables tested - a Hercules-branded unit (`USB\VID_12BA&PID_00FF`, "Licensed by Sony Computer Entertainment America") and a generic/OEM C-Media-chipset unit (`USB\VID_0D8C&PID_0134`, "USB PnP Audio Device"). Both are mono USB Audio Class capture devices and both are fully functional at the OS/host level.

Summary

Two separate, sequential bugs were found while trying to run Rocksmith 2014 Edition Remastered on Linux via Steam Play (Proton):

1. **Launch crash (SIGSYS)** on current-generation Proton builds (Proton 11.0, Proton Experimental, GE-Proton11-1), caused by a `madvise` syscall being killed by the seccomp sandbox of Steam Linux Runtime 4.0. Worked around by forcing an older Proton build (Proton 10.0 / GE-Proton10-34) that uses Steam Linux Runtime 3.0 ("Sniper") instead.

2. **Real Tone Cable never detected**, even once the game launches successfully - confirmed on the calibration screen, the in-game Tuner, and the manual audio device picker (Tools > Options > Path/Input), all independently. Extensive follow-up investigation (update 2026-07-18, see Part 2) narrowed this down through several layers:

3. Wine's ALSA-backed WASAPI capture enumeration silently filters out mono-only USB Audio Class capture devices (both Real Tone Cables are mono; confirmed via a working kernel-level ALSA loopback workaround presenting a fake-stereo version of the signal, which *is* then detected and opened by the game).

4. Even once a working, verified, signal-carrying capture device is opened by the game (proven three independent ways - see Part 2), the in-game level meter never reacts to it, in any screen.
5. A full `WINEDEBUG=+wasapi,+mmdevapi,+alsa` trace shows Wine's own WASAPI/ALSA implementation behaving correctly throughout (continuous real audio reads, correct buffer cycling, zero errors, zero silence flags) - meaning the remaining defect is inside Rocksmith2014.exe's own closed-source signal-detection logic, most likely a WASAPI capture timing/padding semantics difference between Wine and real Windows that the game's engine does not tolerate.

This document exists so the underlying Wine/Proton (and possibly Ubisoft engine-side) bugs can be reported and reproduced by developers. A separate, more detailed technical report focused specifically on Part 2's later findings (with full trace evidence) is available on request.

Part 1: Launch crash (SIGSYS)

Symptom

The game process starts and is killed within 2-3 seconds, every time, before any window is created. Steam's content log shows repeated `App Running` -> `Fully Installed` transitions within seconds.

Diagnosis

- `systemd-coredump` / `journalctl` show the 32-bit `Rocksmith2014.exe` process (running under `wine-preloader`) terminated by **SIGSYS (signal 31)**.
- Reproduced on: Proton - Experimental, Proton 11.0, GE-Proton11-1.
- Core dump analysis (`coredumpctl dump` + `eu-readelf -n` on the core file's `PRSTATUS` note) shows:
- `si_signo: 31`, `si_code: 1` (`SYS_SECCOMP`) - the process was killed by a seccomp filter, not a normal crash.
- `orig_rax: 28` - on x86_64 this is the `madvise` syscall number.
- Registers confirm 64-bit execution context (`cs: 0x33`) at the time of the kill.
- The kill happens inside the **Steam Linux Runtime 4.0** container (appid `4183110`, pressure-vessel sandbox), on a completely ordinary `madvise` call made by the wine-preloader during early process setup.
- `steam steam://validate/4183110` confirms the runtime's files are intact - this is not a corrupted install.

- No SELinux AVC denials around the crash (`ausearch -m avc`).
- No `SystemCallFilter=` directive on the systemd user unit that launches Steam (`app-steam@autostart.service`) - the block is not systemd hardening.

Conclusion

Steam Linux Runtime 4.0's seccomp sandbox is killing a plain `madvise()` call from the wine-preloader on this kernel (`7.1.3-ogc3.4.fc44`). This looks like an incompatibility between a very recent kernel and the runtime's syscall allow-list (or an argument value in a newer `madvise` flag that predates the runtime's filter), reproducible across every Proton build that depends on Runtime 4.0.

Workaround

Any Proton build whose `toolmanifest.vdf` requires the older **Steam Linux Runtime 3.0 "Sniper"** (appid `1628350`) instead of Runtime 4.0 does **not** hit this crash:

- **Proton 10.0 (official Valve build)** - works consistently, no crash across repeated testing including after a full game reinstall
- GE-Proton10-34 - same underlying Sniper runtime, but observed to crash with the same SIGSYS on multiple separate occasions (both during initial testing and again after a game reinstall) - **not reliable**, avoid
- Proton 11.0 / Experimental / GE-Proton11-1 - all require Runtime 4.0, all crash consistently

Applied fix: set Steam's per-game compatibility tool (`CompatToolMapping` in `config.vdf`, or Properties -> Compatibility in the Steam UI) to **official Proton 10.0 specifically** (`proton_10`) - not GE-Proton10-34. Note that a game reinstall/update can silently reset this mapping back to GE-Proton10-34 or another default; worth re-checking after any reinstall.

Part 2: Real Tone Cable not detected

With the launch crash worked around, the original problem remained: the in-game calibration screen always shows *"It sounds like your cable is unplugged. Connect a Rocksmith Real Tone Cable."*, regardless of Proton version, ini settings, or which of two physical Real Tone Cables was used.

Host-level audio: confirmed working

- The adapter is correctly enumerated by the kernel and PipeWire/ALSA as `alsa_card.usb-Hercules_Rocksmith_USB_Guitar_Adapter-00` (`Rocksmith Guitar Adapter Mono`, 1 channel, 48kHz).
- It is set as the default PipeWire/ALSA capture source.
- A direct `arecord -D default ...` capture test confirms the OS-level audio pipeline correctly targets this device (source state transitions to `RUNNING` during capture).

This rules out any hardware, cable, or OS/PipeWire configuration issue.

Game-side: audio engine and detection logic

The game ships `RocksmithConfiguration.pdf` (official Ubisoft PC configuration and FAQ, dated 2016-12-10) documenting `Rocksmith.ini` audio options. Setting `DumpAudioLog=1` produces `audiodump.txt` in the game's install directory, which reveals the internal audio engine:

- The game uses **Wwise** (Audiokinetic) with a **PortAudio** sink/source (`pSinkPortAudio`), confirmed by log lines like `pSinkPortAudio->Init() == AK_Success`.
- Input detection goes through `NSoundInputMgr_Win32::DetectInputDevices_PortAudio`, which in every test run logs:

```
*************
NSoundInputMgr_Win32::DetectInputDevices_PortAudio - no new input devices, stop
```

i.e. PortAudio's device enumeration (via its WASAPI host API on Windows/Wine) returns **zero capture devices**, every time, regardless of configuration.

Ini settings tested (per official Ubisoft documentation) - no effect on detection

Setting	Values tried	Result
`ExclusiveMode`	1 (default) -> 0	No change
`Win32UltraLowLatencyMode`		No change

Setting	Values tried	Result
	1 (default) -> 0	
`ForceWDM`	0 (default) -> 1	Value is read correctly by the game (confirmed in log), but does not fix detection
`ForceDirectXSink`	0 (default) -> 1	**Value is silently ignored** - `audiodump.txt` always logs `INIT ForceDirectXSink 0` regardless of the ini file's actual content or combination with `ForceWDM`. Appears to be a dead/vestigial setting in the Remastered build.
`RealToneCableOnly`	left at 0	N/A

Wine-level investigation

- **WINEDEBUG tracing** with channels `+dsound,+mmdevapi,+wdmaud,+winmm,+alsa,+pulse,+hid,+setupapi,+plugplay,+dinput` across multiple runs: mmdevapi trace output contains only `render_*` (playback) calls - **never a single `capture_*` call**. The game/PortAudio never even reaches a successful capture-client open.
- Confirmed via `setupapi`/`hid` traces that Wine *does* correctly enumerate the device's HID interface (`HID\VID_12BA&PID_00FF&MI_02\...`) through `WINEBUS`/`dinput` - this is a red herring; the official Ubisoft documentation confirms the cable should be read as a standard Windows **Recording device** (WaveIn/WASAPI), not via HID.
- **Wine registry inspection**: the capture endpoint for this device exists in Wine's registry (`HKLM\Software\[Wow6432Node\]Microsoft\Windows\CurrentVersion\MMDevices\Audio\Captur` hardware ID `USB\VID_12BA&PID_00FF\...`), but with `"DeviceState"=dword:00000004` (`DEVICE_STATE_NOTPRESENT`) instead of `1` (`DEVICE_STATE_ACTIVE`). Manually forcing this to `1` via `wine reg add` had no effect on PortAudio's enumeration result.

- **ALSA vs PulseAudio Wine driver** (`HKCU\Software\Wine\Drivers\Audio`):
- With `winealsa.drv`: winecfg's own Audio tab "Voice input device" dropdown only offers generic `(System default)` / `In: default` entries - no named devices at all.
- With `winepulse.drv`: the same dropdown correctly lists all capture devices **by name**, including **"Rocksmith Guitar Adapter Mono"**.
- Despite winepulse.drv's enumeration clearly working (proven via winecfg's UI), PortAudio's WASAPI enumeration inside the running game **still** logged `no new input devices, stop` with the pulse driver active.

This last point is the key finding: **winecfg's device list and the game's PortAudio/WASAPI enumeration go through different code paths**, and the one the game actually uses (`IMMDeviceEnumerator::EnumAudioEndpoints(eCapture, ...)` under Wine's mmdevapi) fails to return this device regardless of the underlying Wine audio driver.

Root cause (as understood 2026-07-16)

Wine/Proton's WASAPI capture-endpoint enumeration (`mmdevapi`) does not surface this USB audio capture device to applications, regardless of Proton version (10.0, 11.0, Experimental) or Wine audio driver (ALSA or PulseAudio). Output/render endpoints enumerate and work correctly and consistently; capture endpoints do not, specifically for this class of device (mono, USB Audio Class, `VID_12BA&PID_00FF`).

This is a Wine/Proton-level limitation, not something fixable via Rocksmith's own settings, ini file, or registry edits.

Update 2026-07-18: deeper investigation, root cause refined significantly

Follow-up testing (game was reinstalled after the refund decision above was reconsidered) found that the picture above, while correct as far as it went, was incomplete. Switching Wine's audio driver to ALSA changed PortAudio's enumeration behavior in a way not seen during the original 2026-07-16 testing:

Devices are enumerable after all - but mono capture devices are filtered

With `Software\Wine\Drivers\Audio` set to `alsa` (via `protontricks 221680 sound=alsa`), `audiodump.txt` started showing **non-zero** enumeration results: `NSoundInputMgr_Win32::DetectInputDevices_PortAudio` found 2 devices - an onboard motherboard USB audio chip (stereo, unrelated hardware) and a `default` pseudo-device - and successfully opened both as input slots. **The actual Real Tone Cable was still never listed.**

Comparing ALSA hardware capabilities directly (`/proc/asound/cardX/streamY`):

Device	Capture channels
Real Tone Cable (either one)	**1 (mono)**
The device PortAudio *did* find	2 (stereo)

This strongly suggests **Wine's ALSA-backed WASAPI capture enumeration filters out mono-only USB Audio Class capture devices**, only surfacing stereo-capable ones. Since Real Tone Cables are inherently mono, this alone explains the invisibility seen in the original 2026-07-16 testing.

Confirmed via a working fake-stereo workaround

To test this directly, a kernel-level ALSA loopback device was created (`sudo modprobe snd-aloop`), which creates a genuine ALSA *hardware* card (distinct from a user-space config file). A background `ffmpeg` process duplicated the cable's live mono signal into this loopback as 2 channels, independently verified working via direct `arecord`/`ffmpeg -af astats` tests (both channels carrying identical, real, non-silent signal).

This fake-stereo device was detected and opened by the game - unlike an earlier attempt using a purely user-space ALSA PCM definition (`~/.asoundrc`), which the game's process (sandboxed inside the **Steam Linux Runtime "Sniper" container**, appid `1628350`) never saw at all - the container's bundled ALSA libraries apparently don't chain to the host user's `~/.asoundrc`. A genuine kernel-level ALSA hardware card, by contrast, is visible inside the container the same way real USB hardware is.

But the meter still never reacts - to anything, on any device

With the fake-stereo device confirmed opened by the game (visible in its own log as a valid input slot, confirmed via `/proc/asound/.../status` showing the game's own process holding it open in exclusive `RUNNING` state), the in-game meter **still showed zero reaction**, including to a loud, unambiguous synthetic test tone fed directly into it.

This prompted inverting the test: does the meter react to *anything at all*? Three independent, separately host-verified working audio paths were tested live against the game's calibration/Tuner screens:

1. Loud synthetic test tone into the fake-stereo loopback device - **no reaction**.
2. Live signal from the real cable via the system's actual default audio input (confirmed via PipeWire's own session manager showing an **active** connection from the game's process directly to the cable's capture port) - **no reaction**.
3. Loud ambient noise directed at the (separate, stereo, unrelated) onboard motherboard audio device that the game treats as its first/primary input slot - **no reaction**.

This was also confirmed to be a game-wide issue, not specific to the first-launch calibration gate: the in-game **Tuner** (Tools menu) and the **Path/Input** device picker (Tools > Options, hold Ctrl) show the exact same "cable is unplugged" failure, completely independently of the calibration screen.

WINEDEBUG trace: Wine's implementation is provably correct

A full `WINEDEBUG=+wasapi,+mmdevapi,+alsa` trace (via `PROTON_LOG=1`/`PROTON_LOG_DIR=...` set in the game's Steam launch options) was captured across a live session. Findings:

- 183,549 `capture_*` mmdevapi calls logged - `GetBuffer`/`ReleaseBuffer` cycle continuously and normally.
- `alsa:alsa_read_data` lines confirm **genuine, continuous reads of real audio data** from the actual mono ALSA device - not mocked/empty reads.
- **Zero** occurrences of `AUDCLNT_BUFFERFLAGS_SILENT`.
- **Zero** `err:mmdevapi` lines anywhere in the trace.
- Property-store queries (device friendly name, audio engine format, endpoint form factor) all resolve without error.

Root cause (refined, 2026-07-18)

The audio pipeline is provably intact end-to-end - physical cable -> ALSA -> PipeWire -> Wine's ALSA backend -> Wine's WASAPI/mmdevapi implementation -> real, continuous, correctly-flagged capture buffers delivered to the game process. This has been independently verified via host tooling and via Wine's own debug trace, across three different capture devices (2 real cables + 1 synthetic loopback), all failing identically.

The remaining, unexplained failure is that **Rocksmith2014.exe's own internal signal-level analysis never registers a valid signal**, in any of the game's audio-level UI screens, despite receiving genuinely valid audio data via a Wine WASAPI implementation that traces as fully correct by every external measure available. The leading hypothesis is a **timing/padding/ position semantics difference** between Wine's WASAPI capture implementation and real Windows (e.g. `IAudioClient::GetCurrentPadding`, buffer position/QPC timestamp reporting, or exclusive-mode stream state transitions) that this specific (older, Wwise/PortAudio-based) game engine is sensitive to in a way that doesn't surface as an explicit Wine-side error - i.e. this now looks like it needs either a Wine-side WASAPI fix or an Ubisoft-side engine fix, not a host/user-side workaround.

Proton/GE-Proton versions tested

Build	Runtime required	Launches?	Cable detected?
Proton 10.0	Sniper (1628350)	Yes	No
Proton 11.0	Runtime 4.0 (4183110)	No (SIGSYS)	N/A
Proton - Experimental	Runtime 4.0 (4183110)	No (SIGSYS)	N/A
GE-Proton11-1	Runtime 4.0 (4183110)	No (SIGSYS)	N/A
GE-Proton10-34	Sniper (1628350)	Inconsistent (crashed with SIGSYS on a later attempt, cause not further diagnosed)	Not reached

---

Suggested next steps for developers

- **Valve/Proton / Steam Linux Runtime team**: investigate why a plain `madvise()` syscall from the wine-preloader is killed by Steam Linux Runtime 4.0's seccomp sandbox on kernel `7.1.3-ogc3.4.fc44`. Reproducible with any Proton build depending on runtime appid `4183110`.
- **Wine project**: two separate leads, both confirmed reproducible:
- `mmdevapi`'s ALSA-backed capture-endpoint enumeration appears to filter out mono-only USB Audio Class capture devices - confirmed via a fake-stereo ALSA loopback workaround that *is* enumerated/opened once presented as 2-channel. Worth checking `dlls/winealsa.drv`'s device enumeration/channel-count handling.
- Even once a valid, signal-carrying capture device is opened by an application, something about Wine's WASAPI capture stream state (leading hypothesis: `GetCurrentPadding`/buffer position/timestamp semantics) appears to differ from real Windows in a way this particular game's engine is sensitive to - despite a full WINEDEBUG trace showing textbook-correct API behavior (continuous real reads, zero errors, zero silence flags) on Wine's side. This is the more significant and harder-to-diagnose of the two issues.
- **Ubisoft**: `ForceDirectXSink=1` in `Rocksmith.ini` does not appear to have any effect in the current Remastered build - the audio engine's debug log always reports it as `0` regardless of the ini file's contents. If this is intentional (dead setting), the PC Configuration and FAQ document should be updated to remove it as a documented fallback. More importantly: given Wine's WASAPI implementation traces as fully correct, the remaining defect looks like it may be inside the game's own audio-engine code path taken under Steam Play/Proton - worth a look from an engine developer with access to source, since this cannot be diagnosed further from outside the binary.

Appendix: reproduction environment

- OS: Bazzite (`ghcr.io/ublue-os/bazzite-nvidia-open:stable`), version `44.20260713`
- Kernel: `7.1.3-ogc3.4.fc44.x86_64`
- GPU: NVIDIA GeForce RTX 3090, driver `610.43.3`
- Steam Linux Runtime 4.0 build: `steamrt4_platform_4.0.20260608.242786`
- Steam Linux Runtime 3.0 "Sniper" build (used by Proton 10.0/GE-Proton10-34): appid `1628350`

- Audio adapter 1: Hercules Rocksmith USB Guitar Adapter, `12ba:00ff`, ALSA card name "Adapter" / PipeWire node "Rocksmith Guitar Adapter Mono"
- Audio adapter 2: generic/OEM Real Tone Cable, C-Media chipset, `0d8c:0134`, ALSA card name "USB PnP Audio Device", also mono capture
- Compat tool used for all 2026-07-18 testing: official Proton 10.0 (`proton_10`)